

Projet Génie Logiciel : Puissance 4 3D

Léontine Hordoir

Rym Zerzouh

Émilien Manceau

Nancy Le Corre

2 Juin 2026

Table des matières

1	Introduction	2
2	Choix d'Implémentation et Architecture	2
2.1	Modélisation du Plateau 3D	2
2.2	Intégration MPI	2
3	Stratégies de jeu et intelligence artificielle	3
3.1	Correction mémorielle et profondeur dynamique	3
3.2	L'heuristique de la "fourchette"	3
3.3	Protocole de tests automatisés	3
4	Retours d'expérience des responsables	3
4.1	Léontine	3
4.2	Rym	3
4.3	Émilien	4
4.4	Nancy	4

1 Introduction

On a conçu et implémenté en C++ un jeu de Puissance 4 en trois dimensions. Le jeu se déroule sur un plateau de 20 bâtonnets répartis sur une surface de 4×5 , d'une hauteur maximale de 4 perles. L'objectif est d'aligner 4 perles de sa couleur dans n'importe quelle direction spatiale.

Au-delà de la mécanique de jeu, l'enjeu majeur de ce projet a résidé dans l'élaboration d'une Intelligence artificielle performante et dans l'intégration du jeu sur une architecture distribuée via la bibliothèque MPI (Message Passing Interface) en vue du tournoi final.

2 Choix d'Implémentation et Architecture

2.1 Modélisation du Plateau 3D

La classe `Plateau3D` est le cœur du jeu. Plutôt que d'utiliser un tableau tridimensionnel complexe, nous avons opté pour un tableau unidimensionnel de 80 cases modélisant les emplacements ($4 \times 5 \times 4$).

Pour fiabiliser la structure, nous avons dû procéder à une révision géométrique stricte afin de corriger une inversion critique entre la longueur et la largeur. Ce réalignement des axes a sécurisé le calcul des alignements.



2.2 Intégration MPI

Pour connecter notre IA à l'arbitre distant, nous avons restructuré la classe `JoueurMPI`. Elle prend désormais un pointeur vers notre IA. À chaque tour, elle reçoit le coup adverse, met à jour sa grille locale, sollicite le "cerveau" pour générer une réponse, et l'envoie sur le réseau.

3 Stratégies de jeu et intelligence artificielle

L'algorithme de calcul MinMax a été optimisé par l'intégration d'un élagage Alpha-Beta, couplé à une heuristique d'ordonnancement des coups. En analysant prioritairement les cases situées au centre du plateau, l'IA écarte plus rapidement les branches infructueuses, pour réduire le temps de calcul.

3.1 Correction mémorielle et profondeur dynamique

Lors des simulations, un coup gagnant figeait le plateau, privant l'IA de ses capacités pour les calculs suivants. La correction du système de retrait des pions réinitialise parfaitement le plateau. De plus, le moteur intègre une gestion dynamique de sa profondeur de réflexion, rendant sa difficulté configurable (par défaut fixée à 4).

3.2 L'heuristique de la "fourchette"

La fonction d'évaluation a été affinée pour intégrer la notion de double menace (fourchette). Le programme attribue un bonus massif (ex : +50000) lorsque l'IA crée deux lignes de trois pions ouvertes simultanément, et applique un malus défensif lorsque l'adversaire s'apprête à faire de même.

3.3 Protocole de tests automatisés

Pour valider ces implémentations scientifiquement, un mode de simulation "silencieux" a été développé. Il exécute instantanément des centaines de parties en arrière-plan. En appliquant une alternance stricte du joueur de départ, nous avons pu mesurer avec précision le gain tactique : l'IA Profondeur 4 l'emporte de manière systématique contre l'IA Profondeur 2, confirmant nos choix algorithmiques.

4 Retours d'expérience des responsables

4.1 Léontine

Mon rôle a été de concevoir la structure globale du projet en POO. Ce n'était pas évident au départ de se projeter en 3D et de définir comment les classes allaient interagir entre elles. J'ai découpé les tâches de manière logique : la structure d'un côté, l'IA de l'autre.

4.2 Rym

Je me suis occupée du moteur et de l'IA. Après avoir corrigé des bugs bloquants sur les dimensions de la grille et le MinMax, j'ai intégré l'élagage Alpha-Beta et la tactique des "fourchettes". Enfin, j'ai créé un banc de test silencieux pour prouver en quelques secondes que notre IA profondeur 4 écrasait la profondeur 2.

4.3 Émilien

Je me suis occupé de mettre en place l'environnement Doxygen. Ma mission principale, c'était de m'assurer que pour chaque fonction, chaque paramètre et chaque classe soit rigoureusement commenté. J'ai instauré une charte de nommage très stricte.

4.4 Nancy

J'ai d'abord géré le problème lié à la profondeur 4, puis j'ai modifié le makefile pour structurer la compilation. Ensuite, je me suis attaqué à l'intégration du réseau MPI, pour faire communiquer le jeu avec l'interface, j'ai développé et adapté les fichiers `arbitre`, `TestJeu` et `JoueurMPI`. J'ai également réalisé le rapport et le support pour l'oral.